CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Subject: DAA

Subject code: 24CSH-282

Assignment No.: 3

Student Name: Krish Latawa

UID: 24BAI70541

Branch: BE CSE AIML

Section/Group: 24 AIT-KRG G1

Submitted to: Prof. Mohammad Shaqlain

Date of Submission: 24/04/2026

Design & Analysis of Algorithms

Backtracking Assignment

Repository: DAA GitHub Repo	**Folder:** Assignment	**Deadline: 24/04/2026**

Question 1: N-Queens Problem

Problem Statement

Place N queens on an N x N chessboard such that no two queens attack each other. Two queens attack each other if they are in the same row, column, or diagonal.

Tasks

- Print all possible solutions for any given N
- Count the total number of valid configurations
- Optimize using column and diagonal hashing (set-based tracking)

Approach & Algorithm

Algorithm: Backtracking with Hash Set Optimization

The idea is to place queens row-by-row. For each row, we try every column and check if placing a queen is safe. Instead of scanning the entire board (O(N) check), we use three hash sets to track attacked columns and diagonals in O(1) time.

Key Insight — Diagonal Encoding:

- Main diagonal (/): All cells on the same '/' diagonal share the same value of (row - col)
- Anti-diagonal (\): All cells on the same '\' diagonal share the same value of (row + col)

Solution Code (Python)

```
def solve_n_queens(n):
    solutions = []
    cols = set()          # attacked columns
    diag1 = set()         # attacked '/' diagonals (row - col)
    diag2 = set()         # attacked '\' diagonals (row + col)
    board = []

    def backtrack(row):
        if row == n:
            solutions.append(board[:])
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            # Place queen
            cols.add(col)
```

```
            diag1.add(row - col)
            diag2.add(row + col)
            board.append(col)
            backtrack(row + 1)
            # Remove queen (backtrack)
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)
            board.pop()

    backtrack(0)
    return solutions

def print_board(solution, n):
    for col in solution:
        print(' . ' * col + ' Q ' + ' . ' * (n - col - 1))
    print()

def main():
    n = 4
    solutions = solve_n_queens(n)
    print(f'N-Queens Solutions for N = {n}:')
    print(f'Total valid configurations: {len(solutions)}\n')
    for i, sol in enumerate(solutions):
        print(f'Solution {i + 1}:')
        print_board(sol, n)

main()
```

Sample Output (N = 4)

```
N-Queens Solutions for N = 4:
Total valid configurations: 2

Solution 1:
 .  Q  .  .
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .

Solution 2:
 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .
```

Complexity Analysis

Metric	Without Optimization	With Hash Set (Optimized)
Time Complexity	O(N!) with O(N) safety check	O(N!) with O(1) safety check
Space Complexity	O(N^2) for board	O(N) for hash sets
Safety Check Cost	O(N) per placement	O(1) per placement

Question 2: Hamiltonian Cycle

Problem Statement

Given an undirected graph represented as an adjacency matrix, determine whether a Hamiltonian Cycle exists. A Hamiltonian Cycle is a closed loop that visits every vertex in the graph exactly once before returning to the starting vertex.

Tasks

- Print one valid Hamiltonian cycle if it exists
- Return false and print a message if no such cycle exists
- Use adjacency matrix representation for the graph

Approach & Algorithm

Algorithm: Backtracking on Graph

We build the path one vertex at a time using backtracking. At each step, we try to extend the current path by adding an unvisited adjacent vertex. If all vertices are visited and the last vertex connects back to the first, a Hamiltonian cycle is found.

Safety Checks at each step:

- The candidate vertex must be adjacent to the last vertex in the current path
- The candidate vertex must not already be in the path (not yet visited)
- If the path length equals N, verify the last vertex is adjacent to the starting vertex (closing the cycle)

Solution Code (Python)

```
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        # Adjacency Matrix
        self.graph = [[0] * vertices for _ in range(vertices)]

    def add_edge(self, u, v):
        self.graph[u][v] = 1
        self.graph[v][u] = 1   # undirected graph

    def is_safe(self, v, path, pos):
        # Check if vertex v is adjacent to previous vertex
        if self.graph[path[pos - 1]][v] == 0:
            return False
        # Check if vertex already in path
        if v in path:
            return False
        return True

    def hamiltonian_util(self, path, pos):
        if pos == self.V:
            # Check if last vertex connects back to start
            return self.graph[path[pos - 1]][path[0]] == 1

        for v in range(1, self.V):
            if self.is_safe(v, path, pos):
                path[pos] = v
```

```
                if self.hamiltonian_util(path, pos + 1):
                    return True
                path[pos] = -1   # backtrack

        return False

    def hamiltonian_cycle(self):
        path = [-1] * self.V
        path[0] = 0   # Start from vertex 0

        if not self.hamiltonian_util(path, 1):
            print('No Hamiltonian Cycle exists')
            return False

        print('Hamiltonian Cycle found:')
        print(' -> '.join(str(v) for v in path) + ' -> ' + str(path[0]))
        return True

# --- Test Case 1: Cycle exists ---
g1 = Graph(5)
g1.add_edge(0, 1); g1.add_edge(0, 3)
g1.add_edge(1, 2); g1.add_edge(1, 3); g1.add_edge(1, 4)
g1.add_edge(2, 4); g1.add_edge(3, 4)
print('Test Case 1:')
g1.hamiltonian_cycle()

# --- Test Case 2: No cycle ---
g2 = Graph(5)
g2.add_edge(0, 1); g2.add_edge(0, 2)
g2.add_edge(1, 2); g2.add_edge(2, 3)
print('\nTest Case 2:')
g2.hamiltonian_cycle()
```

Sample Output

```
Test Case 1:
Hamiltonian Cycle found:
0 -> 1 -> 2 -> 4 -> 3 -> 0

Test Case 2:
No Hamiltonian Cycle exists
```

Complexity Analysis

Metric	Value
Time Complexity	O(N!) in the worst case
Space Complexity	O(N) for the path array
Graph Representation	Adjacency Matrix — O(V^2) space
Adjacency Check	O(1) using matrix lookup